SLGA™ Software Lifecycle Governance Architecture — Software Lifecycle Governance Layer

SLGA™ Software Lifecycle Governance Architecture — Software Lifecycle Governance Layer

--------- Forwarded message ---------

From: Knowenger <knowenger@gmail.com> Date: Fri, Oct 16, 2026, 9:11 AM Subject: SLGA™ Software Lifecycle Governance Architecture — Software Lifecycle Governance Layer To: Miroslav Pis <swissstaffingagency@gmail.com>, <Gutflex@gmail.com>, < originopenfoundation@gmail.com>

SLGA™ Software Lifecycle Governance Architecture — Software Lifecycle

Governance Layer Official Software Lifecycle Governance Architecture for Software Products, Applications, Components, Services, Platforms, AI Systems, Autonomous Software Agents, Digital Infrastructure, Embedded Software, and Future Software Systems

SLGA™ serves as the official OOF® Software Lifecycle Governance

architecture used for Software Requirements governance, Software Creation governance, Software Dependency governance, Software Configuration governance, Software Quality governance, Software Security governance, Software Release governance, Software Deployment governance, Software Change Management governance, Software Retirement governance, and cross-cutting Software Lifecycle Control governance across software products, applications, components, services, platforms, AI systems, autonomous software agents, digital infrastructure, embedded software, and future software systems.

SLGA™ does not prescribe a software development methodology, replace software engineering, perform software creation, determine software quality or security outcomes without applicable evidence, execute releases or deployments, authorize actions independently of legitimate authority, replace operational management, or substitute for specialized technical, contractual, regulatory, safety, or security requirements. SLGA™ defines the governance architecture required to ensure that software becomes purpose-defined, requirements-grounded, creation-traceable, dependency-aware, configuration-controlled, quality-assessed, security-governed, release-authorized, deployment-verifiable, change-controlled, legitimately retired, and continuously governable throughout its complete lifecycle.

SLGA™ consists of 11 Core Parent Standards and 55 Core Modules forming the

primary Software Lifecycle Governance backbone within the OOF® ecosystem.

Additional compatible standards, bridge standards, and specialized

Software Lifecycle Governance standards may extend the architecture when new Software Lifecycle Governance spaces are identified. However, the standards listed below form the official core Software Lifecycle Governance structure of SLGA™. Canonical Definition

SLGA™ — Software Lifecycle Governance Architecture is the OOF® reference governance architecture for governing identifiable software throughout its lifecycle, from Software Requirements and Creation through Dependencies, Configuration, Quality, Security, Release, Deployment, Change Management, and Retirement, with Software Lifecycle Control operating as a cross-cutting governance layer preserving lifecycle state consistency, legitimate transitions, bounded exceptions, control continuity, and verifiable control effectiveness. Core Question

How can software remain identifiable, controllable, and governable throughout its lifecycle when its artifacts, dependencies, conditions, actors, authority relationships, and operational states continuously change? Governance Flow

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Cross-Cutting Governance Layer: Software Lifecycle Control

Core Parent Standards 1. Software Requirements Standard (SRS)

Governed Space: Software Requirements Governance

Core Question: How can software purpose, requirements, scope, changes, and traceability remain identifiable and governable throughout the software lifecycle?

• Software Creation Standard (SCS)

Governed Space: Software Creation Governance

Core Question: How can software creation remain identifiable, methodologically controlled, attributable, and traceable across human, automated, and autonomous creation processes?

• Software Dependency Standard (SDS)

Governed Space: Software Dependency Governance

Core Question: How can software dependencies remain identifiable, origin-traceable, suitable, change-controlled, and governable throughout the software lifecycle?

• Software Configuration Standard (SCFS)

Governed Space: Software Configuration Governance

Core Question: How can software configurations, builds, artifacts, and versions remain identifiable, controlled, reproducible where required, and traceable across lifecycle changes?

• Software Quality Standard (SQS)

Governed Space: Software Quality Governance

Core Question: How can software quality requirements, evaluation criteria, testing coverage, defects, and resulting quality conditions remain governed through identifiable evidence?

• Software Security Standard (SSS)

Governed Space: Software Security Governance

Core Question: How can software security requirements, exposures, testing, vulnerabilities, and remediation remain identifiable, controlled, and verifiable throughout the software lifecycle?

• Software Release Standard (SRLS)

Governed Space: Software Release Governance

Core Question: Under what governed conditions can an identifiable software release candidate become ready, legitimately authorized, recorded, and subject to withdrawal where required?

• Software Deployment Standard (SDPS)

Governed Space: Software Deployment Governance

Core Question: How can software deployment remain planned, authorized, operationally identifiable, monitored, and recoverable across actual deployment environments?

• Software Change Management Standard (SCMS)

Governed Space: Software Change Management Governance

Core Question: How can software changes remain requested, impact-assessed, coordinated, routed through applicable lifecycle governance, and legitimately closed?

• Software Retirement Standard (SRTS)

Governed Space: Software Retirement Governance

Core Question: How can software be legitimately, safely, and verifiably retired when its operational use, support obligations, dependencies, data, users, authority relationships, and residual responsibilities may continue beyond the decision to discontinue it?

• Software Lifecycle Control Standard (SLCS)

Governed Space: Software Lifecycle Control Governance

Core Question: How can software remain identifiable, controllable, and governable throughout its lifecycle when its states, transitions, actors, dependencies, operational conditions, authority relationships, and governance requirements continuously change? Compatible OOF® Architectures

SLGA™ interoperates with multiple OOF® architectures governing

complementary spaces surrounding software and its lifecycle.

GOA™ — Governance Architecture provides the constitutional governance

layer defining how governance itself is structured, bounded, delegated, validated, and maintained.

ORA™ — Operational Reality Architecture governs what is actually occurring in Operational Reality, while SLGA™ governs the lifecycle conditions under which identifiable software is created, controlled, released, deployed, changed, and retired.

AGA™ — Accountability Governance Architecture governs responsibility,

accountability, attribution, authority, and related accountability structures, while SLGA™ governs their application to software lifecycle functions without replacing general accountability governance.

AIG® — AI Governance Architecture governs AI behavioral integrity and

behavioral governance, while SLGA™ governs the software lifecycle of AI-enabled software without becoming an architecture for AI behavior itself.

ASGA™ — Autonomous Systems Governance Architecture governs

autonomous-system identity, authority, execution, constraints, coordination, intervention, escalation, and operational governability, while SLGA™ governs the lifecycle of software used by autonomous systems and autonomous software agents.

ADIT® — Continuous Audit & Evidence Governance Architecture governs

evidential observation, preservation, correlation, verification, examination, and auditability, while SLGA™ defines the software lifecycle governance conditions to which relevant evidence applies.

VALIDOS™ — Validation Governance Architecture governs whether a

validation basis sufficiently supports a Validation Determination, while SLGA™ governs the lifecycle requirements, controls, and decisions that may depend on validated information.

INTEGROS® — Integrity Governance Architecture governs integrity across

objects, states, evidence, and processes, while SLGA™ governs software lifecycle integrity requirements within its distinct lifecycle domain.

OBIDENITY® — Origin-Bound Identity Governance Architecture governs

Origin, Identity, Ownership, Authority, Permission, Responsibility, Provenance, Transfer, Continuity, and Auditability, while SLGA™ governs identifiable software objects and their lifecycle relationships.

SIMULOS® — Simulation Governance Architecture governs simulation purpose,

structure, execution, outputs, and lifecycle, while SLGA™ governs software lifecycle conditions and does not treat simulated software behavior as verified operational behavior by default.

SLGA™ serves as the common Software Lifecycle Governance architecture

supporting the broader OOF® ecosystem by ensuring that software remains a governed lifecycle object rather than a collection of disconnected development, testing, release, deployment, maintenance, and retirement activities.
Architectural Position

SLGA™ governs the complete Software Lifecycle Governance Lifecycle from

software purpose and requirements formation through creation, dependency and configuration control, quality and security governance, release, deployment, change management, and retirement, while preserving continuous

lifecycle control across all stages.

The architecture progresses through:

Software Requirements → Software Creation → Software Dependencies →

Software Configuration → Software Quality → Software Security → Software Release → Software Deployment → Software Change Management → Software Retirement

Software Lifecycle Control operates across the complete flow rather than

as an additional operational stage after Retirement.

Together these standards govern why software is required, how it is created, what dependencies it uses, which configurations and artifacts exist, what quality and security conditions apply, when release is legitimate, how deployment is controlled, how changes re-enter applicable governance, when retirement may occur, and whether control remains consistent and effective across the complete lifecycle.

SLGA™ remains distinct from the operational activities, technical tools,

organizational arrangements, and external requirements through which software is developed or operated.

Human Development / Automated Creation / Autonomous Software Generation /

Software Modification / Deployment Activity / Operational Event → Software Lifecycle-Relevant Inputs → Governed Software Lifecycle under SLGA™

Software Created ≠ Software Governed.

Software Released ≠ Software Deployed.

Software Operational ≠ Lifecycle Control Verified.

SLGA™ governs the software lifecycle — not software engineering execution

itself. Core Governed Software Lifecycle Spaces

SLGA™ establishes eleven distinct governable Software Lifecycle spaces:

Software Requirements

→ why software exists, what it must achieve, which requirements apply, how its scope is defined, and how requirements remain traceable through change.

Software Creation

→ how software creation methods, creators, automated creation processes,

software artifacts, and development traceability remain governed.

Software Dependencies

→ what components and dependencies software relies upon, where they originate, whether they are suitable, and how their changes and continuity are controlled.

Software Configuration

→ how software baselines, builds, artifacts, versions, configuration

changes, and reproducibility conditions remain identifiable and governed.

Software Quality

→ what quality requirements and criteria apply, how testing coverage and

defects are governed, and what quality condition can be established from available evidence.

Software Security

→ what security requirements apply, which exposures and vulnerabilities

exist, how security testing is controlled, and how remediation remains governed.

Software Release

→ when a release candidate is ready, who may authorize it, what release record is established, and when withdrawal may be required.

Software Deployment

→ how deployment is planned, authorized, connected to actual production versions, monitored, and subject to rollback or recovery.

Software Change Management

→ how changes are requested, impact-assessed, coordinated, returned to applicable lifecycle governance, and closed.

Software Retirement

→ how retirement is planned, support is ended, migration is governed, residual dependencies are resolved or retained, and retirement closure is verified.

Software Lifecycle Control

→ how lifecycle states remain consistent, transitions remain legitimate,

exceptions remain bounded, control remains continuous across transfers, and lifecycle controls remain assessable and effective.

Each space answers a different methodological question.

Each can fail independently.

Their separation is therefore foundational to SLGA™. Cross-Architecture Boundary Lock

SLGA™ is intentionally bounded to prevent duplication with adjacent OOF® architectures and specialized software-related systems. SLGA™ ↔ GOA™

GOA™: governs the constitutional structure of governance.

SLGA™: governs the software lifecycle under applicable governance

authority and boundaries.

General Governance Authority ≠ Software Lifecycle Authorization. SLGA™ ↔ ORA™

ORA™: What is actually occurring in Operational Reality?

SLGA™: What software lifecycle state, requirements, controls, and

decisions apply to that operational reality?

Declared Software State ≠ Actual Operational State. SLGA™ ↔ AGA™

AGA™: governs responsibility, accountability, attribution, authority, and

related accountability structures.

SLGA™: governs software lifecycle control functions and their applicable

responsibility and authority conditions.

Responsibility Assigned ≠ Lifecycle Control Effective. SLGA™ ↔ ADIT®

ADIT®: governs evidence and auditability.

SLGA™: governs software lifecycle requirements, states, transitions, and

controls supported by relevant evidence.

Evidence Recorded ≠ Lifecycle Requirement Satisfied. SLGA™ ↔ VALIDOS™

VALIDOS™: governs Validation Determinations.

SLGA™: governs software lifecycle decisions and controls that may require validated inputs.

Validation Determination ≠ Release Authorization. SLGA™ ↔ INTEGROS®

INTEGROS®: governs integrity across objects, states, evidence, and

processes.

SLGA™: governs software lifecycle conditions and the controls required to

preserve them.

General Integrity Governance ≠ Complete Software Lifecycle Governance. SLGA™ ↔ OBIDENITY®

OBIDENITY®: governs Origin-Bound Identity and related ownership,

authority, provenance, and continuity relationships.

SLGA™: governs software objects and their lifecycle identity,

configuration, and state relationships.

Software Identity ≠ Software Lifecycle State. SLGA™ ↔ AIG®

AIG®: governs AI behavior.

SLGA™: governs the software lifecycle of AI-enabled systems.

AI Behavioral Governance ≠ Software Lifecycle Governance. SLGA™ ↔ ASGA™

ASGA™: governs autonomous execution, authority, constraints,

intervention, and operational governability.

SLGA™: governs software lifecycle requirements and controls applicable to

autonomous software systems.

Autonomous Execution Capability ≠ Software Lifecycle Authority. SLGA™ ↔ SIMULOS®

SIMULOS®: governs simulations and simulated conditions.

SLGA™: governs actual software lifecycle objects, decisions, and states.

Simulated Deployment ≠ Actual Deployment.

Simulated Control Success ≠ Operational Control Effectiveness. Architectural Origin

SLGA™ evolved from the foundational methodological problem that software is frequently managed through separate technical activities without a sufficiently unified governance structure across its complete lifecycle.

Traditional processes may proceed:

Requirements → Development → Testing → Release → Deployment → Maintenance

→ Retirement

while software identity, dependencies, configurations, authority, evidence,

lifecycle states, and control responsibilities remain fragmented across tools and organizations.

This becomes increasingly inadequate where:

software is generated by automated or autonomous systems

the creator differs from the operator

software depends on external components with independent lifecycles

a build differs from the artifact actually deployed

a release is authorized but a different configuration enters production

quality verification and security verification have different outcomes

a technically deployable version lacks legitimate deployment authorization

multiple software versions operate simultaneously

changes affect requirements, dependencies, configurations, and previously

established assessments

an autonomous agent can execute tools beyond its legitimate lifecycle authority

software responsibility transfers without corresponding authority or

capability

documented controls fail during actual operation

exceptions accumulate and become routine

retirement is declared while dependencies, credentials, data, or obligations remain active

or:

software lifecycle records no longer correspond to actual operational

reality

SLGA™ therefore establishes software as an identifiable governed lifecycle

object.

Its foundational lifecycle is:

Requirements → Creation → Dependencies → Configuration → Quality →

Security → Release → Deployment → Change Management → Retirement

with:

Lifecycle State Consistency → Lifecycle Transition Control → Lifecycle

Exception Management → Lifecycle Control Transfer & Continuity → Lifecycle Control Assessment

operating as the cross-cutting Software Lifecycle Control mechanism. Architectural Structure

Architecture: SLGA™ — Software Lifecycle Governance Architecture

Category: Governance & Enforcement

Subcategory: Software Lifecycle Governance

Core Parent Standards: 11

Core Modules: 55

Architecture Model: Lifecycle-Based · Modular · Methodology-Neutral ·

Technology-Neutral

Primary Governance Domain: Software Lifecycle Governance

The architecture deliberately separates governable spaces that can independently fail.

Software may:

• be created without sufficiently defined requirements

• be generated without identifiable creation authority or traceability

• incorporate dependencies with unknown origin or suitability

• possess configurations that cannot be reliably identified

• be evaluated against incomplete quality criteria

• contain unresolved security exposures

• be released without sufficient readiness or authorization

• be deployed in a version or configuration different from the

• change without returning to materially affected lifecycle governance

• be retired while dependencies or continuing obligations remain active

• possess conflicting declared and actual lifecycle states

• undergo transitions without legitimate authority

• operate under expired or unauthorized exceptions

• transfer between control holders without preserving operational

• or appear governed through documented controls that do not function in

authorized release; capability; practice.

SLGA™ treats these not as one generic software management problem, but as

distinct Software Lifecycle Governance spaces requiring distinct methodological controls. Complete Lifecycle Principle

SLGA™ does not treat software creation, release, or deployment as completion of the software lifecycle.

The complete lifecycle may proceed:

Software Purpose → Requirements → Creation → Dependencies → Configuration

→ Quality → Security → Release → Deployment → Change Management → Retirement

Software may continue materially after release and deployment.

During its lifecycle:

Requirements may change.

Software artifacts may change.

Dependencies may change.

Configurations may change.

Quality conditions may change.

Security conditions may change.

Release authorization may change.

Deployment states may change.

Operational responsibility may transfer.

Exceptions may arise or expire.

Controls may become ineffective.

Retirement obligations may continue after operational withdrawal.

The lifecycle may therefore proceed:

Deployed Software → Change Request → Impact Assessment → Applicable

Lifecycle Re-Entry → Updated Verification → Release / Deployment where applicable → Continued Operation

and eventually:

Retirement Planning → End-of-Support Management → Migration where applicable → Residual Dependency Management → Retirement Closure

Software Release ≠ Lifecycle Completion.

Software Deployment ≠ Lifecycle Completion.

Operational Retirement ≠ Elimination of Continuing Governance Obligations. Recursive Lifecycle Principle

SLGA™ does not assume that software progresses through its lifecycle only once.

A software change may affect requirements.

A dependency update may affect security.

A configuration change may affect quality verification.

A security remediation may require a new build.

A release may be withdrawn.

A deployment may require rollback.

A retired component may require renewed governance because a material residual dependency remains.

SLGA™ therefore supports targeted recursive governance.

The canonical lifecycle re-entry rule is:

Re-enter the earliest materially affected Software Lifecycle Governance

space.

The canonical reassessment rule is:

Reassess what materially changed. Preserve what remains valid.

Lifecycle re-entry does not require restarting every software lifecycle stage by default.

Only materially affected governance spaces and dependent downstream

conditions require renewed governance.

Software Change ≠ Complete Lifecycle Restart by default.

Lifecycle Re-Entry ≠ New Software Identity by default.

Configuration Change ≠ New Software Origin by default.

Re-entry must preserve historical traceability. Software Identity & Configuration Principle

SLGA™ does not assume that one software name represents one stable operational object.

A software product may contain:

multiple components

multiple versions

multiple builds

multiple configurations

multiple deployment instances

multiple environments

multiple dependency structures

or:

multiple simultaneous operational states

Therefore:

Software Product ≠ Software Version.

Software Version ≠ Software Build.

Software Build ≠ Deployment Instance.

Release Candidate ≠ Authorized Release.

Authorized Release ≠ Verified Production Deployment.

Declared Lifecycle State ≠ Verified Lifecycle State.

Each relevant software object and state must remain identifiable within its applicable governance context. Authority & Control Principle

Software lifecycle activities may be performed by:

Humans

Teams

Organizations

External Providers

Automated Systems

AI Systems

Autonomous Agents

or:

Distributed Human–Machine Arrangements

The actor may change without eliminating the need for legitimate lifecycle governance.

SLGA™ preserves the distinction:

Technical Capability ≠ Legitimate Authority.

Task Authority ≠ Constituent Action Authority.

Responsibility Assigned ≠ Control Capability Established.

Control Assigned ≠ Control Effective.

A system's ability to create, modify, release, deploy, or retire software

does not independently establish authority to perform those actions. Continuity & Retirement Principle

Software lifecycle governance must remain historically reconstructable from requirements formation through retirement and continuing obligations.

Changes in:

Software Version

Configuration

Dependency

Environment

Creator

Operator

Control Holder

Authority

Lifecycle State

Release

Deployment

Retirement Status

or:

Technical Identifier

must not silently destroy the continuity of the governed software lifecycle.

Therefore:

Current Software State ≠ Complete Software Lifecycle History.

System Migration ≠ New Software Origin by default.

Control Transfer ≠ Historical Accountability Erasure.

Retirement Decision ≠ Retirement Completion.

Operational Withdrawal ≠ Residual Dependency Resolution.

Retirement Closure ≠ Historical Evidence Erasure.

Retirement establishes an appropriate lifecycle disposition for software.

It does not establish that the software never existed or that all continuing obligations have disappeared. Foundational Architecture Principles

SLGA™ governs the Software Lifecycle — not software engineering execution

itself.

Software Created ≠ Software Governed.

Software Purpose ≠ Verified Requirements.

Requirements Documented ≠ Requirements Traceable.

Software Generated ≠ Creation Authority Established.

Automated Creation ≠ Autonomous Authority.

Component Identified ≠ Dependency Suitable.

Dependency Available ≠ Dependency Continuity Guaranteed.

Software Product ≠ Software Version.

Software Version ≠ Software Build.

Software Build ≠ Deployment Instance.

Configuration Recorded ≠ Configuration Verified.

Build Reproducibility ≠ Software Quality.

Testing Performed ≠ Quality Established.

Quality Assessment ≠ Security Assessment.

Vulnerability Identified ≠ Vulnerability Remediated.

Security Remediation Applied ≠ Security Effectiveness Verified.

Release Candidate ≠ Authorized Release.

Release Authorized ≠ Deployment Authorized.

Release Authorized ≠ Actual Deployment Verified.

Deployment Executed ≠ Deployment Verified.

Deployment Successful ≠ Lifecycle Control Effective.

Change Requested ≠ Change Authorized.

Change Implemented ≠ Change Closed.

Software Change ≠ Complete Lifecycle Restart by default.

Lifecycle Re-Entry ≠ New Software Identity by default.

Retirement Decision ≠ Retirement Completion.

End of Support ≠ End of Operational Use.

Software Migration ≠ Residual Dependency Elimination.

Operational Retirement ≠ Elimination of Continuing Governance Obligations.

Declared Lifecycle State ≠ Verified Lifecycle State.

Technical Transition Capability ≠ Legitimate Transition Authority.

Transition Executed ≠ Transition Verified.

Exception Requested ≠ Exception Approved.

Exception Authorized ≠ Requirement Eliminated.

Exception Approved ≠ Residual Risk Resolved.

Responsibility Assigned ≠ Authority Established.

Authority Established ≠ Capability Available.

Control Transferred Administratively ≠ Control Continuity Verified.

Control Documented ≠ Control Implemented.

Control Implemented ≠ Control Effective.

Control Activity Recorded ≠ Control Objective Achieved.

Agent Instruction ≠ Enforced Authority Boundary.

Task Authority ≠ Constituent Action Authority.

Current Software State ≠ Complete Software Lifecycle History. Canonical Principle

SLGA™ does not prescribe software development methodologies, replace software engineering, perform software creation, execute testing, release, deployment, change, or retirement operations, independently grant lifecycle authority, replace specialized security or quality requirements, govern general accountability, perform general evidence governance, or substitute for contractual, regulatory, safety, technical, or other domain-specific decision systems. SLGA™ governs the methodological conditions under which identifiable software becomes requirements-grounded, creation-traceable, dependency-aware, configuration-controlled, quality-assessed, security-governed, release-authorized, deployment-verifiable, change-controlled, legitimately retired, and continuously governed through consistent lifecycle states, authorized transitions, bounded exceptions, preserved control continuity, and evidence-supported control effectiveness.

Software may be created by humans or autonomous systems, may operate through multiple versions and configurations, may depend on external components, may undergo repeated lifecycle re-entry, may transfer between responsible actors, may remain operational under exceptional conditions, and may retain dependencies or obligations after retirement. SLGA™ provides the governance architecture for that entire lifecycle while preserving the distinction between declared governance and verified operational control.

SLGA™ governs Software from Requirements to Retirement — with continuous

control across the entire lifecycle.